DevOps Shack
devopsshack.com | @devopsshack

30 Days of GitLab

Advanced GitLab DevOps Series

Complete Syllabus — 30 Videos · 4 Weeks · 1 Final Project

30	4	1	100%
Videos	Weeks	Final Project	Hands-On

The Complete Pipeline — Built Incrementally

Week	Days	Focus	What you build
Week 1	Days 1–7	**GitLab Platform Foundations**	Projects, groups, MRs, code review, container registry
Week 2	Days 8–14	**GitLab CI/CD Core**	Runners, stages, variables, artifacts, rules, debugging
Week 3	Days 15–22	**Docker + Security + Kubernetes**	SAST → Docker → Trivy → Vault → K8s → DAST → prod
Week 4	Days 23–30	**Advanced, Admin & Hardening**	Review apps, templates, DORA metrics, RBAC, final project

WEEK 1

GitLab Platform Foundations

Goal: Understand GitLab deeply — beyond just Git hosting

Day 1 — GitLab Overview & Architecture

What GitLab actually is under the hood

What is GitLab	Complete DevOps lifecycle platform — source control, CI/CD, security, registry, monitoring in one app
Core Components	Rails (web/API) · PostgreSQL (metadata) · Redis (cache/queues) · Sidekiq (background jobs) · Gitaly (Git ops)
GitLab Editions	CE (free/open-source) · EE (advanced features: DORA, security dashboards) · SaaS (gitlab.com)
SaaS vs Self-Managed	SaaS: no infra, shared runners, always updated | Self-managed: full control, air-gap, VPC, custom integrations

Day 2 — Projects, Groups & Subgroups

Structure that controls CI/CD variable inheritance and runner access

Groups vs Projects	Group = namespace owning projects. Variables & runners defined at group level cascade to all child projects
Subgroup Nesting	Up to 20 levels: company → dept → team → project. CI/CD vars cascade the entire hierarchy
Visibility Levels	Private (members only) · Internal (any logged-in user) · Public (everyone incl. unauthenticated)
Import/Export	Projects export as .tar.gz with repo data, issues, MRs, CI config — useful for instance migrations

Day 3 — Git Workflow & Protected Branches

Branching model determines how CI/CD pipelines trigger

GitFlow vs Trunk	GitFlow: long-lived branches, scheduled releases. Trunk-based: short branches, commit to main daily — CI/CD friendly
Protected Branches	Restrict direct push, require MRs, min approvals, passing CI, signed commits — Settings → Repository → Protected Branches
Branch Naming & CI Triggers	rules: if: $CI_COMMIT_BRANCH == 'main' — feature/* runs lint+test, main runs full deploy pipeline

Day 4 — Merge Requests Deep Dive

MRs are the gate where code review and CI/CD pipeline results meet

MR Lifecycle	Draft → Ready → Approved → Pipeline passes → Merged. Each step is enforceable
Approval Rules	N approvals from: specific users, group members, or CODEOWNERS. Resets when new commits are pushed
Merge Strategies	Merge commit (full history) · Squash (clean history) · Rebase (linear, no merge commit)

CODEOWNERS	Maps file paths to required approvers. /src/auth/* @security-team auto-gates auth changes

Day 5 — Code Review & Collaboration

Review tools that enforce quality without slowing teams down

Inline Suggestions	Propose exact code changes from comments — author applies with one click, creating a commit
Discussion Threads	Threads must be resolved before merge (if enforced) — ensures feedback is addressed, not dismissed
Draft MRs	Prefix title with Draft: — prevents accidental merge. Mark ready only when truly ready

Day 6 — Issues, Boards & Agile Planning

Issues link code, commits and pipelines — everything traces back to work items

Issues & Labels	Scoped labels (type::bug, type::feature) for mutual exclusivity. Powers meaningful board filters
Issue Boards (Kanban)	Columns map to labels. Multiple boards per project — one for engineering, one for management
Milestones	Sprint containers with burndown chart. Closes #42 in MR description auto-closes the issue on merge

Day 7 — Container & Package Registry

Built-in registry — images live next to the code, no DockerHub needed

Container Registry	registry.gitlab.com/<group>/<project> — CI_REGISTRY_USER/PASSWORD auto-injected into every job
Image Tagging Strategy	Tag with $CI_COMMIT_SHORT_SHA (immutable) + $CI_COMMIT_REF_SLUG (branch) + semantic version
Package Registry	Supports npm, Maven, PyPI, NuGet, Helm charts — publish from CI, consume with access control
Cleanup Policies	Keep last N tags, delete older than X days — prevents unbounded storage growth

WEEK 2

GitLab CI/CD Core

Goal: Master pipelines from zero — build a CI pipeline piece by piece

Day 8 — CI/CD Architecture & First Pipeline

Understand the coordinator model before writing a single job

Pipeline Architecture	GitLab creates Pipeline + Job objects. Runners poll API, claim jobs, execute via executor, stream logs
.gitlab-ci.yml Structure	Top-level: stages, variables, default, job defs. A job needs minimum a script: key. Use CI Lint to validate
Executor Types	Shell (host) · Docker (fresh container per job — standard) · Kubernetes (Pod per job — scales to zero)
Pipeline Triggers	push · merge_request_event · schedule · pipeline · api · web — source in $CI_PIPELINE_SOURCE

Day 9 — GitLab Runners — Install, Register & Configure

Specific runners are mandatory for production

Shared vs Specific	Shared = all GitLab.com projects. Specific = registered to one project/group. Use specific in production
Runner Configuration	config.toml: concurrent (max parallel jobs), executor, volumes, network mode. Tag runners for job routing
Runner Security	Never run privileged runners with production access. Protect runners — only run on protected branches/tags

Day 10 — Stages, Jobs, DAG & needs: Keyword

DAG pipelines can cut pipeline runtime by 50%+

Stages & Execution	All jobs in stage N run parallel; stages are sequential. Jobs in same stage DO NOT share state
needs: — DAG Pipelines	Job starts as soon as its needs: dependencies finish — skips waiting for the entire stage
allow_failure	allow_failure: true = pipeline continues marked 'passed with warnings'. Never use on security gates
dependencies: vs needs:	dependencies: controls artifact downloads. needs: controls execution order (+ optional artifacts)

Day 11 — Variables, Secrets & Masking

Predefined variables give you commit, branch, pipeline info for free

Predefined CI Variables	CI_COMMIT_SHA · CI_COMMIT_SHORT_SHA · CI_COMMIT_BRANCH · CI_REGISTRY · CI_ENVIRONMENT_NAME + dozens more
Project & Group Variables	Group vars cascade to all projects/subgroups — set SONAR_TOKEN once, every project gets it
Masked & Protected	Masked = [MASKED] in logs. Protected = only injected on protected branches — use for prod credentials

dotenv Artifacts	Write KEY=VALUE to vars.env, declare reports: dotenv: vars.env — downstream jobs get vars injected

Day 12 — Artifacts & Cache

Confusing the two is a top CI/CD mistake

Artifacts	Files stored after job completion — passed to later jobs or downloaded. Set expire_in to control storage
Artifact Report Types	JUnit (test results in MR) · Coverage · SAST JSON · Container scanning — consumed by GitLab UI
Cache	tar.gz of dirs persisted between pipeline runs on same runner. NOT reliable for critical data
Cache Keys & Policy	key: files: [package-lock.json] = invalidate when lockfile changes. policy: pull in consumer jobs

Day 13 — Rules, Workflow & Conditional Pipelines

rules: replaces only/except — more powerful and readable

rules: Syntax	List of conditions: if / changes / exists / when. First match wins. No match = job excluded
workflow: rules	Top-level: control whether pipeline is created at all — skip [skip ci] commits, empty MR pipelines
Manual Jobs	when: manual + allow_failure: false = hard approval gate. Pipeline pauses until human clicks play
Scheduled Pipelines	Cron-based under CI/CD → Schedules. $CI_PIPELINE_SOURCE == 'schedule' to detect and add nightly jobs

Day 14 — Pipeline Debugging & Optimization

CI_DEBUG_TRACE=true is your best friend when a pipeline fails silently

Reading Job Logs	Timestamped per line: runner setup → script → artifact upload. First non-zero exit code = root cause
CI_DEBUG_TRACE	Enables bash -x inside container: every command printed, every variable expanded
Retry & Interruptible	retry: 2 for flaky network. interruptible: true = cancel older pipeline if newer push arrives (saves minutes)
resource_group	resource_group: production ensures only ONE pipeline deploys to prod at a time — prevents race conditions

WEEK 3

Docker + Security + Kubernetes

Goal: Add Docker build, security scanning, Vault, K8s deploy and DAST in the right order

Day 15 — SAST & Secret Detection

Catch vulnerable code and leaked secrets BEFORE the Docker build

What is SAST	Static analysis: SQL injection, XSS, hardcoded creds, dangerous functions — runs on source, no server needed
GitLab SAST Template	include: template: SAST.gitlab-ci.yml — auto-detects language, runs Semgrep/NodeJS Scan/Gosec/Bandit/SpotBugs
Secret Detection	Scans git history for AWS keys, GitLab tokens, Stripe keys, SSH keys, JWT secrets
MR Security Widget	Vulnerability name, severity (Critical/High/Medium/Low), file + line. Block merges on Critical findings

Day 16 — Building Docker Images — DinD vs Kaniko

Kaniko is the production-correct approach for Kubernetes runners

Docker-in-Docker (DinD)	Requires privileged: true = near root access on host. Acceptable on dedicated VM, not in K8s clusters
Kaniko	Builds images without Docker daemon — unprivileged container. Use gcr.io/kaniko-project/executor:debug
Multi-Stage Dockerfiles	Builder stage (compile) + Runtime stage (minimal image). No build tools in production image — smaller, safer
Image Tagging in CI	$CI_COMMIT_SHORT_SHA (immutable/rollback) + $CI_COMMIT_REF_SLUG (branch latest) + :latest (main only)

Day 17 — Container Scanning with Trivy

Scan the exact image that will be deployed — not just source code

What Trivy Finds	OS packages · language packages inside image · Dockerfile misconfigs · secrets baked into ENV layers
Custom Trivy Job	trivy image --exit-code 1 --severity CRITICAL,HIGH $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
.trivyignore	Version-controlled CVE exception list — documented, auditable, not a way to hide real vulnerabilities

Day 18 — Secrets Management with HashiCorp Vault

CI variables are not a secrets manager — Vault is

Why CI Variables Fall Short	Visible to all devs with project access, not rotated, not audited per-access, exposed in env vars
JWT/OIDC Auth	Job receives short-lived JWT signed by GitLab. Vault validates claims (project, branch, job) — issues token
secrets: keyword	Native GitLab block: declare Vault path + key + CI var name. GitLab handles JWT exchange automatically

Vault Policies	Scoped per project/branch: dev pipelines get dev secrets, prod pipelines get prod secrets — enforced at Vault

Day 19 — Deploying to Kubernetes — kubectl & Helm

First K8s deploy — using Vault credentials, not hardcoded secrets

kubeconfig in CI	Store as masked protected File CI variable — GitLab writes to temp file, sets KUBECONFIG automatically
kubectl apply Deploy	Use --dry-run=client first. Add kubectl rollout status --timeout=3m — fail job if rollout incomplete
Helm in CI	helm upgrade --install --atomic (auto-rollback on failure) --set image.tag=$CI_COMMIT_SHORT_SHA
Namespace per Environment	staging → namespace staging, prod → namespace production. Isolates failing pods across envs

Day 20 — GitLab Kubernetes Agent (agentk) & GitOps

Pull-based GitOps is how mature teams operate

Push-Based Limitations	kubeconfig in CI vars = cluster creds accessible to all devs. Drift possible via manual kubectl apply
agentk — Two Modes	(1) CI/CD tunnel: jobs use agent connection, no kubeconfig in CI (2) GitOps: watches Git, auto-applies manifests
Installing agentk	helm install gitlab-agent with token generated from GitLab UI → Operate → Kubernetes clusters
GitOps Pull Mode	agentk watches .gitlab/agents/<name>/config.yaml manifest_projects path — reconciles cluster to Git state

Day 21 — DAST with OWASP ZAP

Scan the running app after staging deploy — finds what SAST cannot

What DAST Finds	Missing security headers · exposed debug endpoints · unauthenticated APIs · session fixation · clickjacking
Passive vs Active Scan	Passive: observe traffic, no payloads, fast, zero risk. Active: sends attack payloads, comprehensive, slower
GitLab DAST Template	include: template: DAST.gitlab-ci.yml + DAST_WEBSITE: $STAGING_URL — results in MR security widget
Review App + DAST	Deploy review app per MR → run DAST against it → security feedback on every feature branch before merge

Day 22 — Environments, Manual Approval & Deploy to Production

Never auto-deploy to production

GitLab Environments	Tracks: which commit deployed, when, by whom, live URL. Deploy board shows pod status per environment
Protected Environments	Settings → CI/CD → Protected Environments — restrict deploy to Maintainer role only
Manual Approval Gate	when: manual + allow_failure: false between dast and deploy-prod — pipeline pauses for human review
Deploy-Prod & Rollback	Vault prod credentials (main branch scoped) · rollout status check · helm rollback on failure

WEEK 4

Advanced Pipeline, Admin & Hardening

Goal: Review apps, pipeline patterns, DORA metrics, self-hosted GitLab, RBAC

Day 23 — Review Apps & Dynamic Environments

Every MR gets its own live URL — reviewers test the running app

Dynamic Environment Names	environment: name: review/$CI_MERGE_REQUEST_IID — unique namespace per MR in Kubernetes
Auto-Stop on Merge	environment: auto_stop_in: 2 days + on_stop: stop-review — namespace deleted when MR merges
Review App URL in MR	View app button appears in MR widget — one click to live preview. DAST auto-runs against this URL

Day 24 — Multi-Project & Parent-Child Pipelines

When one pipeline triggers another

When to Split	Independent services deploying separately, shared library pipelines, monorepo per-service pipelines
trigger: Multi-Project	trigger: project: group/other-project passes variables downstream — bridge job reflects downstream status
Parent-Child Pipelines	trigger: include: services/frontend/.gitlab-ci.yml — children run parallel, parent waits for all
Passing Variables Downstream	variables: IMAGE_TAG: $CI_COMMIT_SHORT_SHA — available in downstream pipeline as regular CI var

Day 25 — DRY Pipelines — extends, include & Templates

Copy-pasting pipeline config is the fastest way to create inconsistency

extends: Inheritance	.base-deploy (dot hides it) holds common settings. Child jobs override only what differs
include: Sources	local (same repo) · project (another GitLab project — shared CI library) · remote (HTTPS URL) · template (built-in)
YAML Anchors	&anchor / *alias / <<: merge key — inline reuse within same file. Use extends: for cross-file reuse
Shared CI Template Library	Project ci-templates with docker-build.yml, security.yml, k8s-deploy.yml — include: ref: v1.2.0 for versioning

Day 26 — Blue-Green & Canary Deployments

Zero-downtime strategies that catch problems before all users see them

Blue-Green	Two identical environments — deploy to green, smoke test, switch LB instantly. Rollback = switch back to blue
Canary	10% of pods on new version. Monitor error rate + latency for 15 min. Promote to 100% or scale canary to 0
GitLab Pipeline Implementation	deploy-canary + deploy-promote + rollback jobs with resource_group: production on all prod jobs

GitLab Deploy Boards	Environments → production → deploy board: coloured squares per pod — see canary split in real time

Day 27 — DORA Metrics, Pipeline Analytics & Slack Notifications

Measure if your DevOps is actually working

DORA Metrics	Deployment Frequency · Lead Time for Changes · MTTR · Change Failure Rate — auto-computed by GitLab
GitLab DORA Dashboard	Analytics → CI/CD Analytics + Value Stream Analytics + Operate → Environments deployment frequency
Pipeline Analytics	Find bottleneck jobs (longest duration), most-failing jobs (reliability). Combine with DAG to optimise
Custom Slack Notification	curl Slack webhook with commit SHA + author + pipeline URL — rules: on main branch and when: on_failure

Day 28 — GitLab Administration — Install, Backup & Scale

Running GitLab yourself means you own upgrades and backups

Omnibus Installation	Single package: Rails+PostgreSQL+Redis+Sidekiq+Gitaly+Nginx. Configure /etc/gitlab/gitlab.rb, run gitlab-ctl reconfigure
HTTPS & SMTP	external_url + letsencrypt['enable'] = true — auto-issues/renews cert. SMTP for MR notifications + pipeline results
Backup & Restore	gitlab-backup create — includes DB, uploads, artifacts, LFS. gitlab.rb + SSL certs backed up separately
Upgrade Path	Cannot skip major versions. Check upgrade path tool at docs.gitlab.com. Always: backup → read release notes → upgrade one major at a time

Day 29 — RBAC, Permissions & Runner Security Hardening

Now that you understand everything — lock it all down

GitLab RBAC Model	Guest · Reporter · Developer (push non-protected, trigger pipelines) · Maintainer (manage runners, envs) · Owner
Group Role Inheritance	Group role cascades to all projects — override at project level. Inheritance flows down only
Protected Environments	Require 2 Maintainer approvals — even pipelines on main branch are blocked until approvals given
Runner Hardening	Protected mode (protected branches only) · specific tags · no privileged: true · regular token rotation

Day 30 — Final Project: Production-Grade DevSecOps Pipeline

Build the complete 11-stage pipeline end-to-end

build	npm-compile: install deps, compile React frontend, upload dist/ as artifact
test → sast	unit-test (Jest + JUnit XML) · sast-scan (GitLab SAST template) · secret-detect
docker-build → trivy	Kaniko build (no privileged) tagged $CI_COMMIT_SHORT_SHA · Trivy scan — fail on CRITICAL CVEs

deploy-staging → dast	Helm upgrade on staging namespace (Vault JWT secrets) · OWASP ZAP passive scan
approval → deploy-prod → notify	Manual gate → Helm prod deploy + rollback · Slack success + failure notifications